

(ECS708) Report lab8: Density Estimation

Assignment 2

Carlota Cortés Mir ; Student ID: 250711969

Q1 - Produce a plot of $F1$ against $F2$. (You should be able to spot some clusters already in this scatter plot). Comment on the figure and the visible clusters?

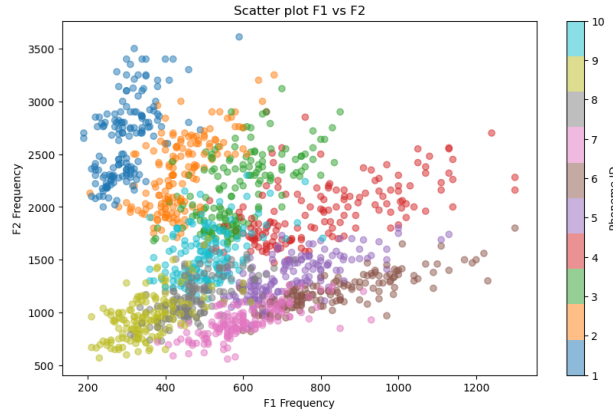


Figure 1: Scatter plot F1 vs F2

The scatter plot (Figure 1) shows the F1 frequency versus the F2 frequency in the Peterson and Barney dataset, where each point is coloured by its phoneme ID (1-10).

Overall, distinct clusters are already visible, each corresponding to a different vowel class.

Many clusters are well separated, for example the case of phoneme ID= 1 (dark blue), phoneme ID= 2 (orange), phoneme id= 3 (dark green) and phoneme id=4 (red). The other clusters are still visible but are more close to each other and some overlap exists, especially in the region of lower F1 and F2 values (phoneme IDs 5-10).

This visible clustering indicates that these two features provide discriminative information and are possibly effective for vowel classification.

Q2 - Run the code multiple times for $K = 3$, what do you observe? Use figures and the printed MoG parameters to support your arguments.

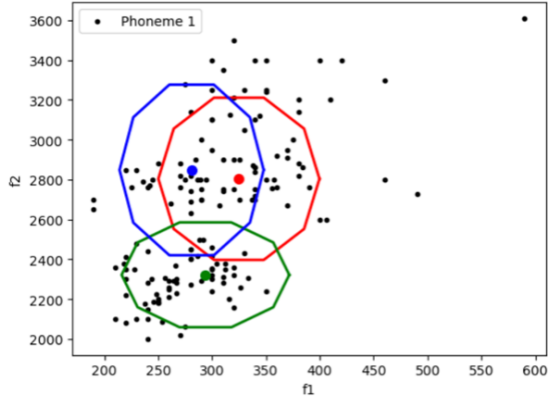
After running the code three times for $K=3$, the final solutions in iteration 100 are similar across runs (Figures 2 and 3. Each run finds three Gaussians that model the same regions, with similar MoG parameters. However, the initial iterations look different, because the starting means are randomly initialized at each run. Therefore, every run begins with a different configuration.

Additionally, the component labels/indices (blue, green, red) change between runs, but the overall fit of the data is the same. The same Gaussians appear but in a different row order in the mean matrix (Table 1). In other words, component 1 in one run may correspond to component 3 in another.

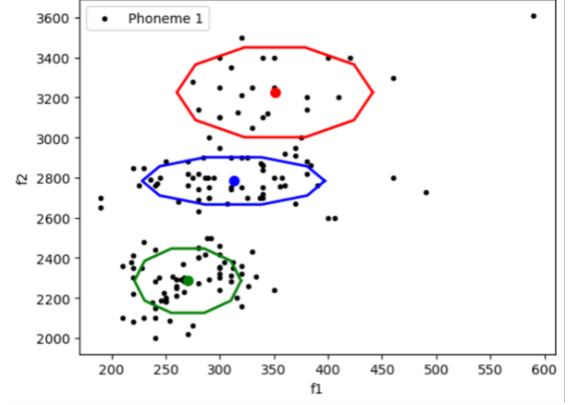
EM only guarantees convergence to a local maximum, not the global. Therefore, different random initializations can lead to different final means. Nevertheless, by observing the dynamic plots, the ellipses change very little after 70 iterations, indicating that the algorithm has converged.

Q5 - Use the 2 MoGs ($K = 3$) learnt in tasks 2 & 3 to build a classifier to discriminate between phonemes 1 and 2, and explain the process in the report

To build a classifier to discriminate between phonemes 1 and 2, I used the two MoG models with $K=3$ trained separately on phoneme 1 and phoneme 2 in the previous tasks. The template already provided how to build the dataset X containing all samples whose ground-truth label is 1 or 2. For each trained phoneme model, I loaded the GMM parameters (μ_1, s_1, p_1) and (μ_2, s_2, p_2) from the saved .numpy files. For every sample x in X , I used the provided `get_predictions`

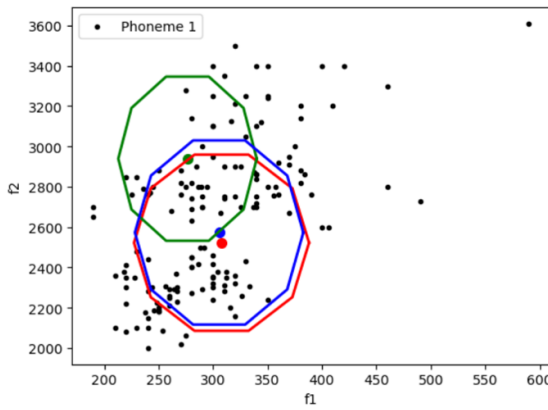


(a) Iteration=1

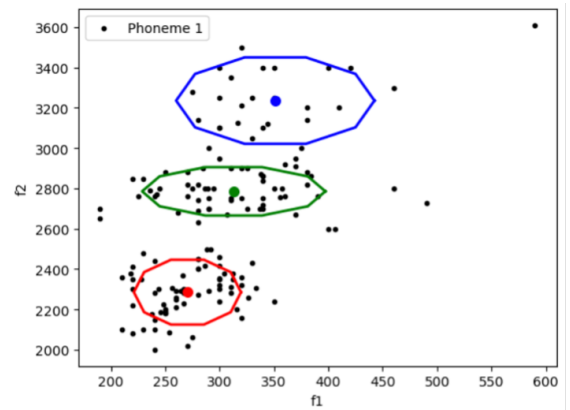


(b) Iteration=100

Figure 2: Evolution of the MoG fit for phoneme 1 with $k=3$ (first initialization)



(a) Iteration=1



(b) Iteration=100

Figure 3: Evolution of the MoG fit for phoneme 1 with $k=3$ (second initialization)

function to compute the component-wise weighted densities Z for each model and summed across components to obtain the total likelihoods. I then applied the Maximum Likelihood decision rule, which assigns x to phoneme 1 if $p(\theta_1) > p(\theta_2)$, and to phoneme 2 otherwise. Finally, I compared the predicted labels with the true labels for all selected samples and computed the accuracy and the misclassification error ($1 - \text{accuracy}$). With $K=3$, the classifier achieved an accuracy of 0.964 and a misclassification error of 0.036, indicating that the two MoG provide a good fit for the $F1$ - $F2$ distributions of phonemes 1 and 2.

Q6 - Repeat for $K = 6$ and compare the results in terms of accuracy.

For $K=6$ instead of $K=3$, the classifier achieved an accuracy of 0.957 and a misclassification error of 0.043, slightly worse than with $K=3$, suggesting that increasing the model complexity did not improve the classification between phoneme 1 and 2.

Q7 - Display a "classification matrix" assigning labels to a grid of all combinations of the $F1$ and $F2$ features for the $K = 3$ classifiers from above. Next, repeat this step for $K = 6$ and compare the two.

For $K=3$ (Figure 4a), the decision boundary between phoneme 1 and 2 is smooth and simple. It captures the main separation between the red and green clusters. For $K=6$, the decision boundary is similar but becomes more flexible, especially in areas where the two overlap. Nevertheless, the overall shape of the separating region is similar, indicating that the discriminative structure is

Execution	Means (F1,F2)	
1	350.84	3226.34
	270.40	2285.47
	312.59	2783.90
2	270.39	2285.43
	312.98	2785.89
	351.22	3236.04
3	270.40	2285.47
	350.84	3226.34
	312.59	2783.90

Table 1: MoG component means for phoneme 1 with K=3 for three different random initializations.

already captured by the simpler K=3 model.

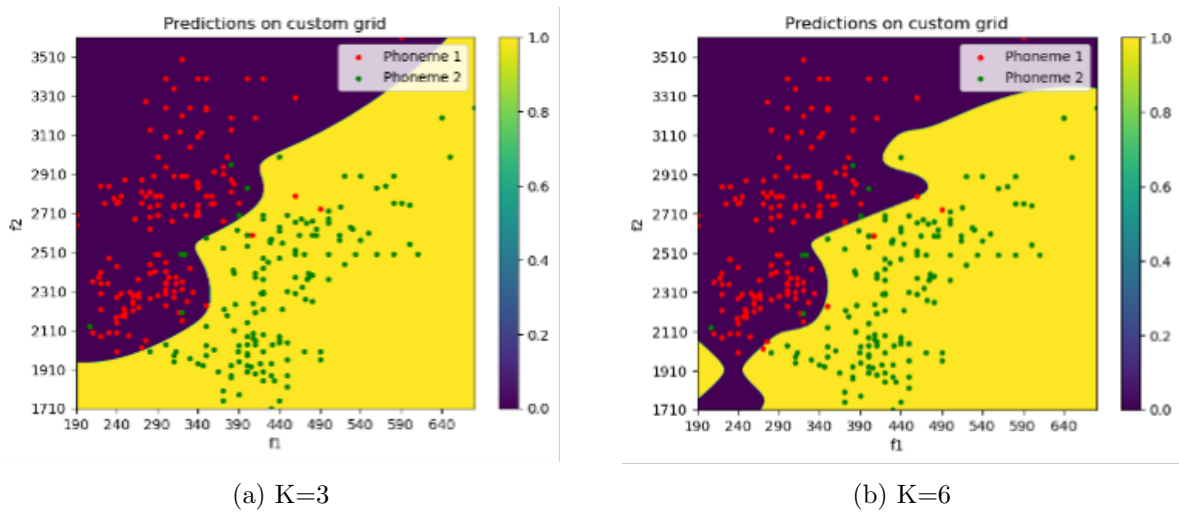


Figure 4: Classification matrix on custom grid

Q8 - Try to fit a MoG model to the new data. What is the problem that you observe? Explain why it occurs

I tried to fit a MoG model to this new data ($X = [F1, F2, F1+F2]$), but after three EM iterations I obtained these errors:

- RuntimeWarning: divide by zero encountered in scalar divide
- ValueError: Input contains infinity or a value too large for dtype('float64')

The EM algorithm fails because the third feature $F1+F2$ is an exact linear combination of $F1$ and $F2$, so the data is linearly dependent in 3D. This makes the covariance matrix singular, which means non invertible and zero determinant. Therefore, a zero determinant causes division by zero and infinite likelihood values.

Q9 - Suggest ways of overcoming the singularity problem and implement one of them. Show any training outputs in the report and discuss.

There's two main ways to overcome the singularity problem (Lecture 8 - Overfitting in ML density estimation):

- Put constraints in the form of the covariance matrix.
- Regularise the covariance matrix by adding a diagonal matrix

I implemented the second option. During the M-step, after computing each covariance $s[i, :, :]$, I added: $s[i, :, :] += \text{epsilon} * \text{np.eye}(D)$. This adds a small value epsilon to each diagonal element

of the covariance matrix and leaves not diagonal elements unchanged, which makes the covariance matrix strictly positive definite and invertible. Therefore, the EM algorithm now runs without errors.

The output shows the fitted MoG in the 3D feature space $[F1, F2, F1+F2]$ (Figure 5).

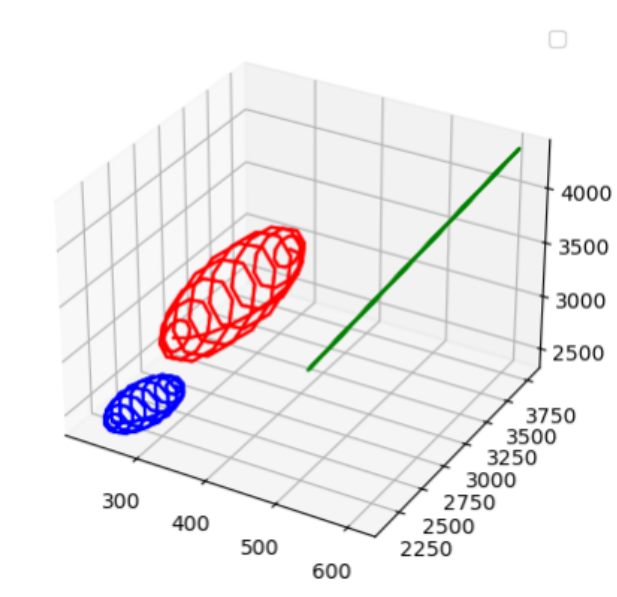


Figure 5: Fitted MoG $[F1, F2, F1+F2]$